МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №1
по курсу «Программирование графических процессов»

Освоение программного обеспечения для работы с технологией CUDA. Примитивные операции над векторами.

Выполнил: Немкова Анастасия Романовна
Группа: М8О-408Б-22
Преподаватели: А.Ю. Морозов, Е.Е. Заяц

Москва, 2025

Условие

Цель работы. Ознакомление и установка программного обеспечения для работы с программно-аппаратной архитектурой параллельных вычислений (CUDA). Реализация одной из примитивных операций над векторами.

Вариант 6. Поэлементное возведение в квадрат вектора.

Входные данные. На первой строке задано число n - размер векторов. На следующей строке записано n вещественных чисел - элементы вектора.

Выходные данные. Необходимо вывести n чисел - результат поэлементного возведения в квадрат исходного вектора.

Программное и аппаратное обеспечение

Графический процессор:

- Модель: NVIDIA GeForce RTX 3050 Laptop GPU
- Compute Capability: 8.6
- Объем графической памяти: 4 294 443 008
- Разделяемая память на блок: 49 152
- Количество регистров на блок: 65 536
- Размер warp: 32
- Максимальное количество потоков на блок: (1024 × 1024 × 64)
- Максимальное количество блоков: (2 147 483 647 × 65 535 × 65 535)
- Объем константной памяти: 65 536
- Количество мультипроцессоров (SM): 16

Центральный процессор:

- Модель: AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz
- Количество ядер: 8
- Количество потоков: 16
- Базовая тактовая частота: 3,2 ГГц
- Максимальная тактовая частота: 4,4 ГГц
- Кеш L1: 64 Кб (на ядро)
- Кеш-память L2: 512 КБ (на ядро)
- Кеш L3: 16 Мб

Оперативная память:

- Тип памяти: DDR4
- Объём памяти: 2 * 8 Гб
- Частота: 3200 Мт/с

Жёсткий диск:

- Тип: SSD
- Модель: SK Hynix HFM001TD3JX013N
- Объём памяти: 1 Тб
- Скорость записи/чтения: 3100/3500 МБ/с

Программное обеспечение:

- OS: Ubuntu 20.04 Linux (WSL2)
- IDE: VS Code
- Compiler: nvcc, gcc

Метод решения

Алгоритм решения задачи заключается в последовательном применении операции возведения в квадрат к каждому элементу входного вектора. Для каждого элемента с индексом i от 0 до n-1 выполняется модификация значения по формуле: arr[i] = arr[i] × arr[i], где исходное значение заменяется на его квадрат.

Описание программы

Типы данных:

- double - основной тип данных для хранения элементов вектора
- int - для хранения размера вектора

Программа реализована в одном файле, содержащем 3 функции:

- Макрос CSC – автоматическая проверка ошибок
- Ядро square_kernel:

```
__global__ void square_kernel(double *arr, int n) {
   int idx = blockIdx.x * blockDim.x + threadIdx.x;
   int offset = blockDim.x * gridDim.x;
   while (idx < n) {
      arr[idx] = arr[idx] * arr[idx];
      idx += offset;
   }
}
```

Принимаем указатель на массив и его размер. Выполняем in-place возведение в квадрат.

- Функция main() - чтение входных данных с проверкой ошибок, управление памятью (CPU/GPU выделение, копирование), запуск ядра, вывод результатов с заданной точностью

Результаты

Были проведены тесты производительности GPU ядра с различными конфигурациями запуска для массивов разного размера (n = 1000, 100 000, 1 000 000, 10 000 000).

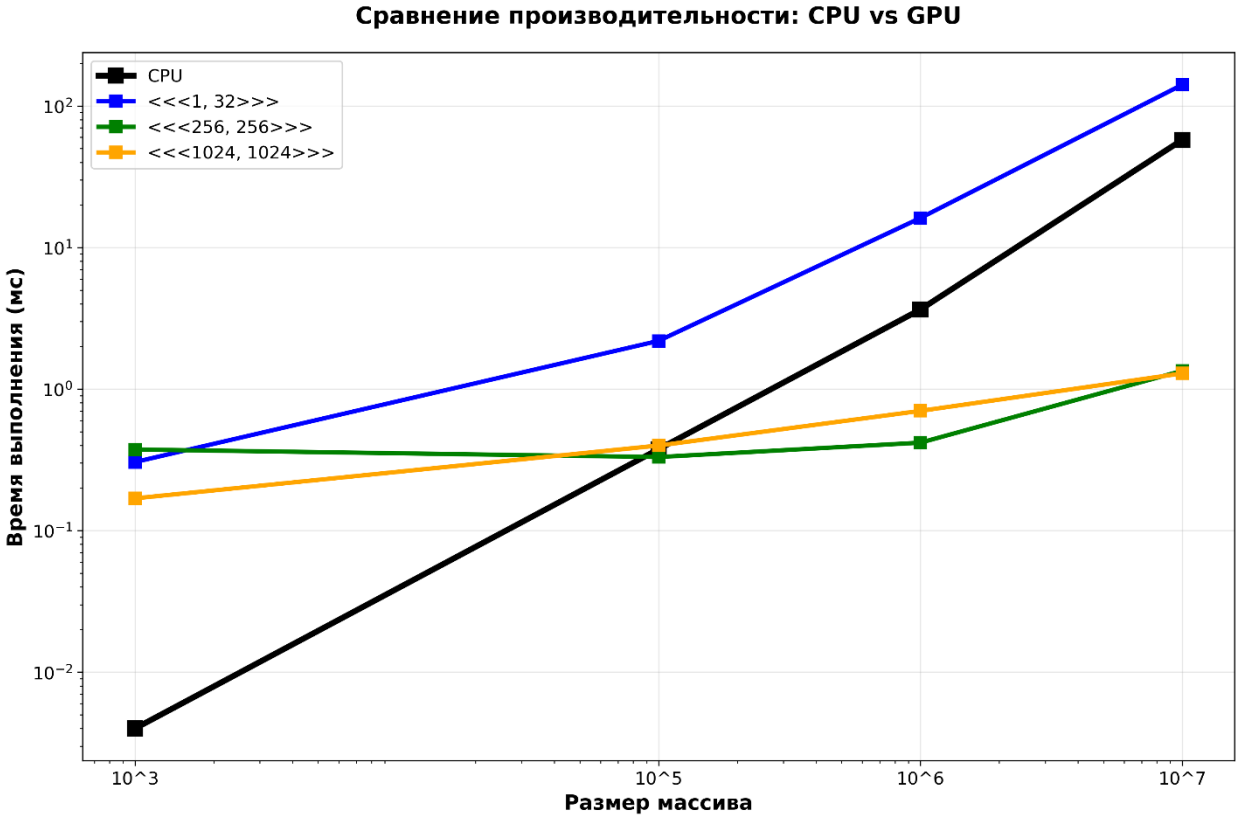
Конфигурация	n = 1000	n = 100 000	n = 1 000 000	n = 10 000 000
<<1, 32>>	0.305152 мс	2.190432 мс	16.161377 мс	141.197662 мс
<<1, 256>>	0.294912 мс	0.687808 мс	3.162976 мс	16.319201 мс
<<16, 256>>	0.460800 мс	0.406048 мс	0.909632 мс	1.577792 мс
<<64, 256>>	0.204800 мс	0.323104 мс	0.513696 мс	1.288096 мс
<<256, 256>>	0.374784 мс	0.331232 мс	0.418816 мс	1.345312 мс
<<1024, 256>>	0.301056 мс	0.330592 мс	0.498272 мс	1.276832 мс
<<1024, 1024>>	0.169984 мс	0.399936 мс	0.701824 мс	1.296608 мс

Также были выполнены замеры работы с использованием CPU

Размер	Время
1000	0.004018 мс
100 000	0.376244 мс

1 000 000	3.646576 мс
10 000 000	57.589821 мс

Ниже представлен график сравнения замеров времени работы GPU с конфигурациями <<1, 32>>, <<256, 256>>, <<1024, 1024>> и CPU на тестах размерами 10^3, 10^5, 10^6 и 10^7



Выводы

Описать область применения реализованного алгоритма. Указать типовые задачи, решаемые им. Оценить сложность программирования, кратко описать возникшие проблемы при решении задачи. Провести сравнение и объяснение полученных результатов.

В ходе данной лабораторной работы было изучено применение технологии CUDA для параллельного выполнения поэлементных операций над большими массивами данных. Также был реализован алгоритм возведения в квадрат элементов вектора.

Полученные результаты демонстрируют, что для малых массивов (n < 10 000) CPU показывает лучшую производительность из-за накладных расходов на организацию параллельных вычислений, однако для больших объемов данных (n > 1 000 000) GPU обеспечивает ускорение благодаря параллелизму.

Анализ различных конфигураций запуска показал, что для массивов средних размеров оптимальная производительность достигается при использовании параметров <<<64, 256>>> и <<<256, 256>>>, а для массивов больших размеров <<1024, 1024>> . Таким образом, правильный выбор конфигурации запуска является важным фактором для достижения максимальной скорости вычислений на GPU.